

CSS

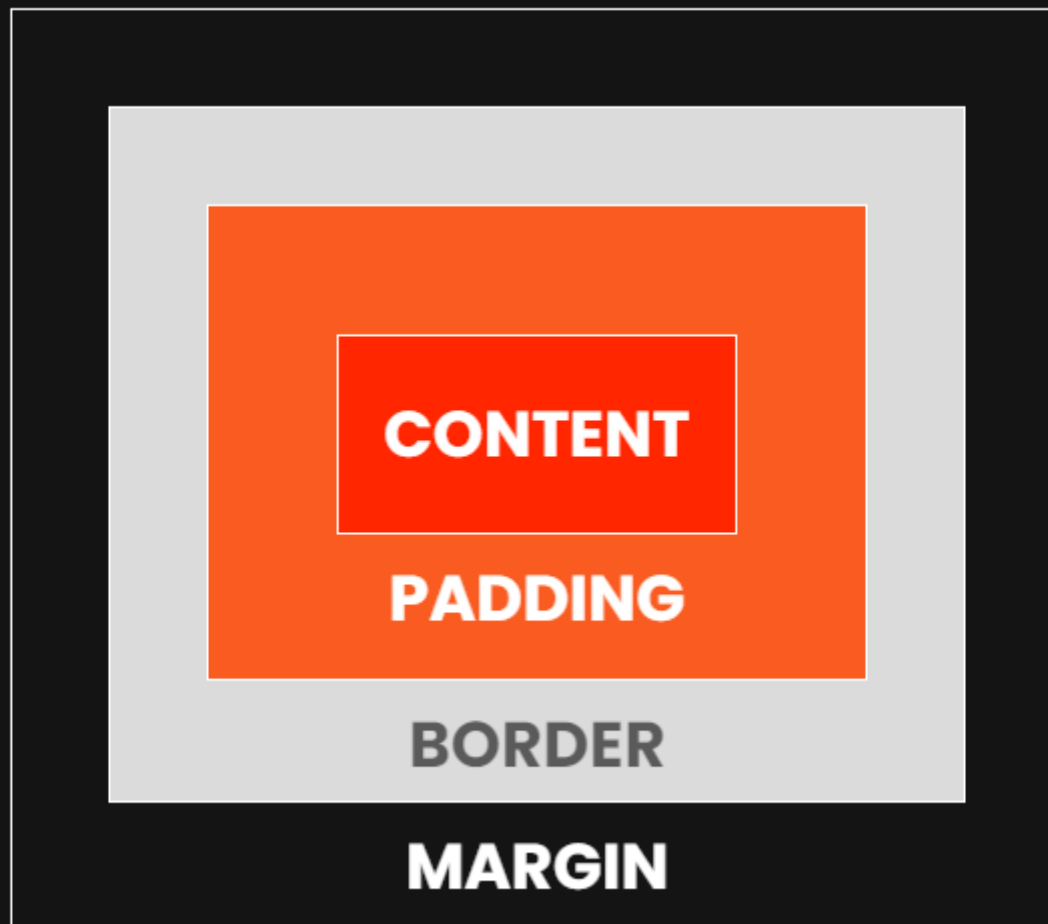
Box Model

Box Model(Modelo de Caja)

El modelo de caja es un concepto fundamental que describe cómo los elementos html son representados y estructurados en una página web.

Cada elemento es tratado como una caja rectangular con diferentes áreas que afectan su tamaño y apariencia.

Box Model (Modelo de Caja)



CONTENT (CONTENIDO)

Es el área donde se muestra el contenido, sean textos, imágenes, etc.

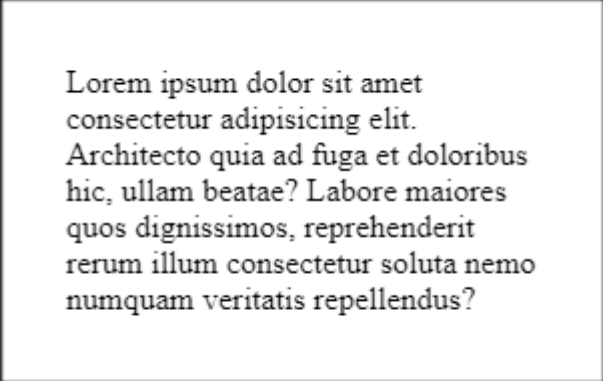
Por defecto ocupa el tamaño del contenido, pero esto puede modificarse con las propiedades `width` y `height` (Ancho y Alto).

Lorem ipsum dolor sit amet consectetur
adipiscing elit. Architecto quia ad fuga et
doloribus hic, ullam beatae? Labore maiores
quos dignissimos, reprehenderit rerum illum
consectetur soluta nemo numquam veritatis
repellendus?

Elemento

PADDING (RELLENO)

Espacio adicional entre el contenido y el borde del elemento. Se puede modificar con la propiedad padding.

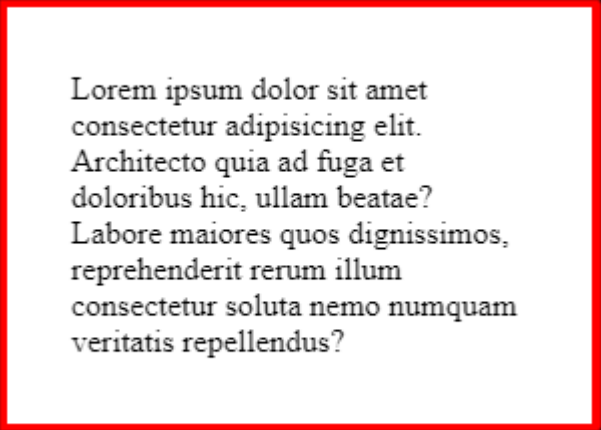


Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et doloribus
hic, ullam beatae? Labore maiores
quos dignissimos, reprehenderit
rerum illum consectetur soluta nemo
numquam veritatis repellendus?

Elemento con padding

BORDER (BORDE)

Es una línea que rodea el elemento junto con su padding(relleno). Se puede modificar mediante la propiedad border.



Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et
doloribus hic, ullam beatae?
Labore maiores quos dignissimos,
reprehenderit rerum illum
consectetur soluta nemo numquam
veritatis repellendus?

Borde color rojo

MARGIN (MARGEN)

Es el espacio entre un elemento y otros. Se puede modificar con la propiedad margin.

Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et
doloribus hic, ullam beatae?
Labore maiores quos dignissimos,
reprehenderit rerum illum
consectetur soluta nemo numquam
veritatis repellendus?

Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et
doloribus hic, ullam beatae?
Labore maiores quos dignissimos,
reprehenderit rerum illum
consectetur soluta nemo numquam
veritatis repellendus?

Elemento sin margen

Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et
doloribus hic, ullam beatae?
Labore maiores quos dignissimos,
reprehenderit rerum illum
consectetur soluta nemo numquam
veritatis repellendus?

Lorem ipsum dolor sit amet
consectetur adipisicing elit.
Architecto quia ad fuga et
doloribus hic, ullam beatae?
Labore maiores quos dignissimos,
reprehenderit rerum illum
consectetur soluta nemo numquam
veritatis repellendus?

Elemento con margen

USO DEL MODELO DE CAJA

Para manejar el tamaño de un elemento tenemos a nuestra disposición las propiedades: `width`, `height`, `padding`, `border` y `margin`. A continuación, pasaremos a aplicarlo a un ejemplo.

```
1  .caja {  
2    background-color: red;  
3    width: 300px;  
4    height: 300px;  
5    padding: 20px;  
6    border: 30px solid white;  
7  }
```

En este ejemplo:

- El contenido mide 300x300 pixeles
- Padding de 20px rodeando el contenido
- Un borde con 30px por los lados.

El Tamaño total del elemento es de 400x400 pixeles (Sumando las propiedades)

BOX-SIZING

Por defecto, CSS para calcular las medidas de un elemento solo incluye su contenido. Esto se puede modificar mediante la propiedad `box-sizing`.

```
1  .caja {  
2    box-sizing: border-box;  
3  }
```

Utilizando `box-sizing: border-box`, el ancho y el alto incluirán el padding y el borde, evitando cálculos adicionales y facilitando el diseño.

WIDTH & HEIGHT

ANCHO Y ALTO DE LOS ELEMENTOS

Descripción

- Las propiedades 'width' y 'height' en CSS se utilizan para definir el ancho y alto de un elemento respectivamente.

Utilización

- Podemos definir el tamaño de un elemento utilizando varias unidades, como por ejemplo px, em, %, vw, etc.
- Se pueden usar valores en porcentajes (%), pero dependen del contenedor padre
- Para hacer elementos responsivos, es mejor usar vw (ancho de la pantalla) y vh (alto de la pantalla).
- Se pueden combinar con max-width y max-height para limitar los tamaños.

Código

Seleccionamos un elemento (En este caso un elemento con clase caja)

```
1  .caja {  
2      height: 240px;  
3      width: 480px;  
4  }
```

Asignamos un height(alto) de 240px y un width(ancho) de 480px.

PADDING

RELLENO DE UN ELEMENTO

Descripción

- La propiedad padding en CSS se usa para agregar espacio interno entre el contenido de un elemento y su borde. Es decir, crea un "acolchado" dentro del elemento.

Utilización

1 Valor

```
1  div {  
2    padding: 20px;  
3  }
```

Con un solo valor se aplica a los 4 lados por igual

2 Valores

```
1  div {  
2    padding: 20px 10px;  
3  }
```

El primer valor será el alto y bajo, El segundo valor para izquierda y derecha

3 Valores

```
1  div {  
2    padding: 20px 10px 5px;  
3  }
```

Primer valor: arriba. Segundo para los lados (Izquierda y derecha). Tercero para el lado de abajo.

4 Valores

```
1  div {  
2    padding: 10px 15px 20px 25px;  
3  }
```

Orden de los lados: arriba, derecha, abajo, izquierda (sentido horario).

```
1  div {  
2    padding-top: 10px;  
3    padding-right: 15px;  
4    padding-bottom: 20px;  
5    padding-left: 25px;  
6  }
```

También podemos definir cada lado individualmente mediante las siguientes propiedades

BORDER

BORDE QUE RODEA UN ELEMENTO

Utilización

```
1 selector {  
2   border: grosor estilo color;  
3 }
```

```
1 div {  
2   border: 2px solid red;  
3 }
```

Esto crea un borde de 2 píxeles, línea sólida, de color rojo alrededor del div.

Descripción

La propiedad border en CSS se usa para agregar un borde alrededor de un elemento HTML. Puedes controlar su grosor, estilo y color, todo en una sola línea o por partes.

Grosor

Podemos especificar el tamaño del grosor del borde, utilizando unidades como px, em, etc.

Estilo

En CSS tenemos ciertos estilos predefinidos para el borde

- Solid (Línea Sólida)
- Dashed (Línea Discontinua)
- Dotted (Línea Punteadas)
- Double (Doble Línea)
- None (Sin borde)

Color

Color: funciona cualquier color válido (red, #333, rgb(0,0,0), etc.)

BORDER

```
1  div {  
2    border-top: 2px solid blue;  
3    border-right: 1px dashed green;  
4    border-bottom: 3px dotted black;  
5    border-left: none;  
6  }
```

También podemos especificar por separado el aspecto del borde según su lado.

Propiedades Individuales

border-width: grosor del borde

border-style: estilo del borde

border-color: color del borde

Border Radius

```
1  div {  
2    border-radius: 8px;  
3  }
```

Con la propiedad border radius podemos redondear los bordes de un elemento.

MARGIN

ESPACIO ENTRE LOS ELEMENTOS

Descripción

- La propiedad margin en CSS se usa para agregar espacio externo alrededor de un elemento, es decir, el espacio entre ese elemento y los que lo rodean.
- A diferencia de padding (que es espacio dentro del borde), margin es el espacio fuera del borde.

Utilización

- Funciona de manera similar al padding, pudiendo especificar el espacio lado por lado o mediante atajos.
- Colapso de márgenes: es un comportamiento especial que ocurre cuando dos márgenes verticales se encuentran, y en lugar de sumarse, se superponen y se usa solo el valor más grande.
- Cuando no colapsan los márgenes:
 - Cuando hay padding, border, o contenido entre los elementos.
 - Márgenes horizontales nunca colapsan, solo verticales.
 - Si el elemento está en flexbox o grid, los márgenes no colapsan como en flujo normal.

Código

Margen de 12px (Aplicado en los 4 lados del elemento)

```
1  div {  
2    margin: 12px;  
3  }
```

Margin: auto ocupa todo el espacio disponible a los costados. Útil para centrar elementos horizontalmente

```
1  div {  
2    margin: auto;  
3  }
```

CSS

Propiedad Display

¿Que es la propiedad Display?

Esta propiedad define cómo se muestra un elemento en la página, es decir, el tipo de caja de renderizado que se aplica a ese elemento.

Dependiendo del valor de display, el comportamiento del elemento (como si se alinea, si ocupa todo el ancho, etc.) cambia.

Display: Block

El elemento se comporta como un bloque. Ocupa todo el ancho disponible y empieza en una nueva línea. Podemos asignarle tamaño con width y height

Display: Block

Elemento Bloque

Elemento Bloque

Ejemplo de elementos: `<div>`, `<h1>`, `<p>`

Display: Inline

El elemento se comporta como en línea. No inicia una nueva línea y solo ocupa el ancho necesario. Las propiedades `width` y `height` no van a tener efecto.

Display: Inline

Elemento Inline

Elemento inline

Ejemplo de elementos: `<span>`, `<img>`, `<a>`

Display: Inline-Block

El elemento se comporta como en línea. No inicia una nueva línea y solo ocupa el ancho necesario. Podemos asignarle un alto y ancho.

Display: Inline-block

Elemento Inline-Block

Elemento inline-Block

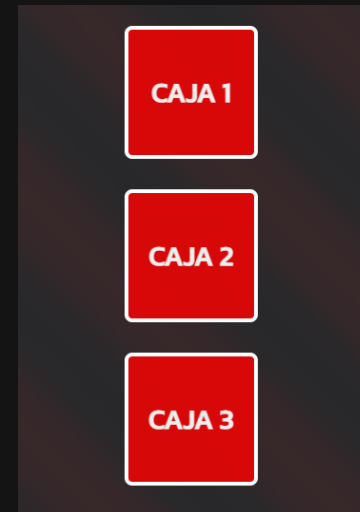
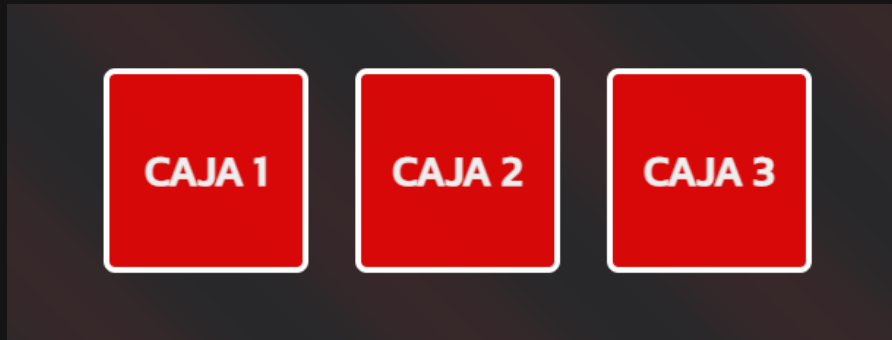
Display: None

El elemento no se muestra en pantalla. Desaparece por completo del flujo del documento.

Display: Flex

La propiedad `display:flex` en CSS se utiliza para crear un contenedor que permite un diseño flexible para sus elementos hijos.

Esta propiedad permite organizar elementos en una fila o columna, y controlar su alineación y espaciado.



Display: Grid

La propiedad `display:grid` en CSS se utiliza para crear un contenedor que permite un diseño basado en filas y columnas.

Esta propiedad nos permite organizar elementos en un grid(Cuadrícula), definiendo la cantidad de filas y columnas que deseemos.



DISPLAY: FLEX

¿Que es Display: Flex?

Es una propiedad de CSS que convierte un contenedor en un contenedor flexible, lo que significa que sus elementos hijos se comportan como "items flexibles" que se pueden alinear y distribuir fácilmente.

¿Cómo se usa?

Primero, debemos aplicar display: flex al contenedor que contiene los elementos que quieres alinear.

```
1 <div class="contenedor">
2   <div>Elemento 1</div>
3   <div>Elemento 2</div>
4   <div>Elemento 3</div>
5 </div>
```

Creamos un elemento contenedor (Con clase)

```
1 .contenedor {
2   display: flex;
3 }
```

Aplicamos la propiedad display: flex

flex-direction

Define la dirección del **eje principal** (por defecto es horizontal, de izquierda a derecha).

row: (Por defecto): Ordena los elementos en fila (Horizontalmente)

row-reverse: En fila, pero al revés

column: Los elementos se ponen en columna (De arriba a abajo)

column-reverse: Columna, pero de abajo hacia arriba.

```
1  .contenedor {  
2    display: flex;  
3    flex-direction: column;  
4  }
```

justify-content

Controla la alineación a lo largo del **eje principal**.

flex-start: (Por defecto): Alineados al comienzo.

flex-end: Alineados al final.

center: Elementos centrados.

space-between: Espacio igual entre los elementos.

space-around: Espacio igual alrededor de los elementos.

space-evenly: Espacio igual entre y fuera de todos los elementos.

```
1  .contenedor {  
2    display: flex;  
3    flex-direction: column;  
4    justify-content: center;  
5  }
```

align-content

Controla la alineación a lo largo del **eje secundario**.

stretch: (Por defecto): Estira los elementos.

flex-start: Alineados al principio.

flex-end: Alineados al final.

center: Elementos centrados.

baseline: Alineados según la línea base del texto.

```
1  .contenedor {  
2    display: flex;  
3    flex-direction: column;  
4    justify-content: center;  
5    align-items: center;  
6  }
```

flex-wrap

Por defecto, todos los elementos intentan estar en una sola línea. Si queremos que se ajusten en varias líneas podemos usar esta propiedad.

nowrap: (Por defecto): Todos los elementos en una línea.

wrap: Los elementos que no entren crean otra línea.

wrap-reverse: Lo mismo que el caso anterior, pero al revés

```
1  .contenedor {  
2    display: flex;  
3    flex-wrap: wrap;  
4  }
```

gap

Distancia entre los items dentro de un contenedor flexbox

```
1  .contenedor {  
2    display: flex;  
3    gap: 16px;  
4  }
```